CifアプリケーションのテンプレートになるServlet

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-055
画面名	CifアプリケーションのテンプレートになるServlet
会社名	
作成日	

概要

概要説明

Cifアプリケーションのテンプレートになるservletの入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	DB2.ＵＤＢ管理ＤＢ、職員情報等（セッション）

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

init

処理概要

Cifアプリケーションのテンプレートになる Servlet で「init」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

doGet

処理概要

Cifアプリケーションのテンプレートになる Servlet で「doGet」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理5：詳細仕様参照）。

1.2. 通常処理：「対象項目がない」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.2.1. 条件成立時：関連情報を取得・設定する（処理6：詳細仕様参照）。

1.2-別.1. 条件不成立時：関連情報を取得・設定する（処理7：詳細仕様参照）。

1-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：関連情報を取得・設定する（処理8：詳細仕様参照）。

doPost

処理概要

Cifアプリケーションのテンプレートになる Servlet で「doPost」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理3：詳細仕様参照）。

1-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：関連情報を取得・設定する（処理4：詳細仕様参照）。

cifServletMain

処理概要

Cifアプリケーションのテンプレートになる Servlet で「cifServletMain」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：「method が等しい FCW_POST」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.1. 条件成立時：関連情報を取得・設定する（処理7：詳細仕様参照）。

1.1.2. 条件成立時：関連情報を取得・設定する（処理8：詳細仕様参照）。

1.1-別.1. 条件不成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

1.1-別.2. 条件不成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1.1-別.3. 条件不成立時：セッションに保持された利用者情報を取得する。

1.2. 通常処理：関連情報を取得・設定する（処理11：詳細仕様参照）。

1-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：関連情報を取得・設定する（処理12：詳細仕様参照）。

mainControl

処理概要

Cifアプリケーションのテンプレートになる Servlet で「mainControl」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連する処理へ制御を渡す。詳細な実行経路は原本対応を参照する。

pdfControl

処理概要

Cifアプリケーションのテンプレートになる Servlet で「pdfControl」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理30：詳細仕様参照）。
2. 関連情報を取得・設定する（処理31：詳細仕様参照）。
3. 繰返し条件「対象項目がない」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理32：詳細仕様参照）。

3.2. 繰返しの各回：「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.2.1. 条件成立時：関連情報を取得・設定する（処理33：詳細仕様参照）。

3.2.2. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3.3. 繰返しの各回：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.3.1. 通常処理：関連情報を取得・設定する（処理34：詳細仕様参照）。

3.3.2. 通常処理：「対象項目がない」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3.3.2.1. 条件成立時：排他制御の範囲内で次の手順を実行する。

3.3.2.1.1. 排他制御の範囲内：関連情報を取得・設定する（処理35：詳細仕様参照）。

3.3.2.2. 条件成立時：「条件36を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.3.2.2.1. 条件成立時：「対象項目」を設定する。

3.3.2.3. 条件成立時：「対象項目」を設定する。

3.3.2-別.1. 条件不成立時：関連情報を取得・設定する（処理37：詳細仕様参照）。

3.3-例外1. 例外条件「FMSystemException e」に該当した場合に次の手順を行う。型名は原本の表記。

3.3-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.3-例外2. 例外条件「FMPropertyException e」に該当した場合に次の手順を行う。型名は原本の表記。

3.3-例外2.1. 例外発生時：関連情報を取得・設定する（処理38：詳細仕様参照）。

3.3-例外2.2. 例外発生時：関連情報を取得・設定する（処理39：詳細仕様参照）。

3.3-例外2.3. 例外発生時：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.3-例外2.3.1. 条件成立時：関連情報を取得・設定する（処理40：詳細仕様参照）。

3.3-例外2.4. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3.3-例外3. 例外条件「IllegalStateException e」に該当した場合に次の手順を行う。型名は原本の表記。

3.3-例外3.1. 例外発生時：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.3-例外3.1.1. 条件成立時：関連情報を取得・設定する（処理41：詳細仕様参照）。

3.3-例外3.2. 例外発生時：関連情報を取得・設定する（処理42：詳細仕様参照）。

3.3-例外3.3. 例外発生時：関連情報を取得・設定する（処理43：詳細仕様参照）。

3.4. 繰返しの各回：排他制御の範囲内で次の手順を実行する。

3.4.1. 排他制御の範囲内：「agent が異なる (SessionAgent)session.getAttribute(AGENTNAME)」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.4.1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4.2. 排他制御の範囲内：関連情報を取得・設定する（処理44：詳細仕様参照）。

3.4.3. 排他制御の範囲内：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.4.3.1. 通常処理：関連情報を取得・設定する（処理45：詳細仕様参照）。

3.4.3.2. 通常処理：「!「対象項目」」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.4.3.2.1. 条件成立時：関連情報を取得・設定する（処理46：詳細仕様参照）。

3.4.3.2.2. 条件成立時：「対象項目」を設定する。

3.4.3-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

3.4.3-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4.4. 排他制御の範囲内：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.4.4.1. 通常処理：「method が等しい FCW_GET、かつ「対象項目」」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3.4.4.1.1. 条件成立時：関連情報を取得・設定する（処理47：詳細仕様参照）。

3.4.4.1.2. 条件成立時：「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.4.4.1.2.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4.4.1-別.1. 条件不成立時：「条件48を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3.4.4.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理49：詳細仕様参照）。

3.4.4.1-別.1.2. 条件成立時：「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.4.4.1-別.1.2.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4.4.1-別.1-別.1. 条件不成立時：関連情報を取得・設定する（処理50：詳細仕様参照）。

3.4.4.1-別.1-別.2. 条件不成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4.4-例外1. 例外条件「FMPrintException e」に該当した場合に次の手順を行う。型名は原本の表記。

3.4.4-例外1.1. 例外発生時：関連情報を取得・設定する（処理51：詳細仕様参照）。

3.4.4-例外1.2. 例外発生時：関連情報を取得・設定する（処理52：詳細仕様参照）。

3.4.4-例外1.3. 例外発生時：関連情報を取得・設定する（処理53：詳細仕様参照）。

3.4.4-例外1.4. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3.4.4-例外2. 例外条件「FMSystemException e」に該当した場合に次の手順を行う。型名は原本の表記。

3.4.4-例外2.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4.4-例外3. 例外条件「FMPropertyException e」に該当した場合に次の手順を行う。型名は原本の表記。

3.4.4-例外3.1. 例外発生時：関連情報を取得・設定する（処理54：詳細仕様参照）。

3.4.4-例外3.2. 例外発生時：関連情報を取得・設定する（処理55：詳細仕様参照）。

3.4.4-例外3.3. 例外発生時：関連情報を取得・設定する（処理56：詳細仕様参照）。

3.4.4-例外3.4. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3.4.5. 排他制御の範囲内：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.4.5.1. 通常処理：関連情報を取得・設定する（処理57：詳細仕様参照）。

3.4.5-例外1. 例外条件「FMSystemException e」に該当した場合に次の手順を行う。型名は原本の表記。

3.4.5-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4.5-例外2. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

3.4.5-例外2.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

3.4.6. 排他制御の範囲内：関連情報を取得・設定する（処理58：詳細仕様参照）。

3.4.7. 排他制御の範囲内：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない・対象項目がない	処理結果を呼出元へ返す
対象項目がない・NOT (session が等しい 未設定)・条件59を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
対象項目がない・NOT (session が等しい 未設定)・条件60を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
対象項目がない・NOT (session が等しい 未設定)・条件61を満たす（原本条件は補足説明を参照）・agent が異なる (SessionAgent)session.getAttribute(AGENTNAME)	処理の異常を通知し、この経路を終了する
対象項目がない・NOT (session が等しい 未設定)・条件62を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・条件63を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
対象項目がない・NOT (session が等しい 未設定)・条件64を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・method が等しい FCW_GET、かつ「対象項目」・対象項目がない	処理の異常を通知し、この経路を終了する

到達条件	結果
対象項目がない・NOT (session が等しい 未設定)・条件65を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・NOT (method が等しい FCW_GET && 「対象項目」)・条件66を満たす（原本条件は補足説明を参照）・対象項目がない	処理の異常を通知し、この経路を終了する
対象項目がない・NOT (session が等しい 未設定)・条件67を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・NOT (method が等しい FCW_GET && 「対象項目」)・条件68を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
対象項目がない・NOT (session が等しい 未設定)・条件69を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・条件70を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
対象項目がない・NOT (session が等しい 未設定)・条件71を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・条件72を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
対象項目がない・NOT (session が等しい 未設定)・条件73を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・条件74を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
対象項目がない・NOT (session が等しい 未設定)・条件75を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・条件76を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
対象項目がない・NOT (session が等しい 未設定)・条件77を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))・条件78を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
対象項目がない・NOT (session が等しい 未設定)・条件79を満たす（原本条件は補足説明を参照）・NOT (agent が異なる (SessionAgent)session.getAttribute(AGENTNAME))	処理結果を呼出元へ返す

handleTimer

処理概要

Cifアプリケーションのテンプレートになる Servlet で「handleTimer」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理6：詳細仕様参照）。

1.2. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.2.1. 条件成立時：関連情報を取得・設定する（処理7：詳細仕様参照）。

1.2.2. 条件成立時：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.2.2.1. 条件成立時：排他制御の範囲内で次の手順を実行する。

1.2.2.1.1. 排他制御の範囲内：関連情報を取得・設定する（処理8：詳細仕様参照）。

1.2.3. 条件成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

1-例外1. 例外条件「IllegalStateException e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：関連情報を取得・設定する（処理10：詳細仕様参照）。

setContentLength

処理概要

Cifアプリケーションのテンプレートになる Servlet で「setContentLength」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 関連情報を取得・設定する（処理7：詳細仕様参照）。
3. 「対象項目がある」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3.1. 条件成立時：関連情報を取得・設定する（処理8：詳細仕様参照）。

3-別.1. 条件不成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

3-別.2. 条件不成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 「対象項目」を設定する。

returnHttpErrorResponse

処理概要

Cifアプリケーションのテンプレートになる Servlet で「returnHttpErrorResponse」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：「対象項目」を設定する。

1.2. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

1.3. 通常処理：関連情報を取得・設定する（処理13：詳細仕様参照）。

1.4. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

1.5. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

1.6. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

1.7. 通常処理：関連情報を取得・設定する（処理17：詳細仕様参照）。

1.8. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

1.9. 通常処理：関連情報を取得・設定する（処理19：詳細仕様参照）。

1.10. 通常処理：関連情報を取得・設定する（処理20：詳細仕様参照）。

1.11. 通常処理：関連情報を取得・設定する（処理21：詳細仕様参照）。

1.12. 通常処理：使用した接続・処理資源を解放する。

1-例外1. 例外条件「IOException e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：関連情報を取得・設定する（処理22：詳細仕様参照）。

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
init	処理1	super.init(config)
init	処理2	super.init(config)
doGet	処理1	errorControl(e,req,res)
doGet	処理2	cifServletMain(req, res, FCW_POST)
doGet	処理3	cifServletMain(req, res, FCW_GET)
doGet	処理4	String fcw = req.getParameter(RETRIEVE)
doGet	処理5	String fcw = req.getParameter(RETRIEVE)
doGet	処理6	cifServletMain(req, res, FCW_POST)
doGet	処理7	cifServletMain(req, res, FCW_GET)
doGet	処理8	errorControl(e,req,res)
doPost	処理1	errorControl(e,req,res)
doPost	処理2	cifServletMain(req, res, FCW_POST)
doPost	処理3	cifServletMain(req, res, FCW_POST)
doPost	処理4	errorControl(e,req,res)
cifServletMain	処理1	errorControl(e,req,res)
cifServletMain	処理2	exitMain()
cifServletMain	処理3	mainControl(req,res,method)

イベント	参照番号	原本条件・処理
cifServletMain	処理4	startMain()
cifServletMain	処理5	Vector data=new Vector()
cifServletMain	処理6	String koteidata[][]={}
cifServletMain	処理7	startMain()
cifServletMain	処理8	mainControl(req,res,method)
cifServletMain	処理9	String koteidata[][]={}
cifServletMain	処理10	Vector data=new Vector()
cifServletMain	処理11	exitMain()
cifServletMain	処理12	errorControl(e,req,res)
pdfControl	処理1	agent.startTimer()
pdfControl	処理2	this.returnHttpResponse(response, agent, result, method)
pdfControl	処理3	agent.startTimer()
pdfControl	処理4	this.returnHttpErrorResponse(response, null, e.getMessage(), code)
pdfControl	処理5	int code = e.getStatusCode()
pdfControl	処理6	agent.startTimer()
pdfControl	処理7	this.returnHttpErrorResponse(response, null, e.getMessage(), code)
pdfControl	処理8	int code = e.getStatusCode()
pdfControl	処理9	result = agent.getLastResponse()

イベント	参照番号	原本条件・処理
pdfControl	処理10	result = agent.overlay()
pdfControl	処理11	String msg = agent.getMessage()
pdfControl	条件12	agent.canOverlay()
pdfControl	処理13	agent.clear()
pdfControl	処理14	agent.parseParameters(request)
pdfControl	処理15	FMResponse result = null
pdfControl	処理16	session.invalidate()
pdfControl	処理17	this.returnHttpErrorResponse(response, null, e.getMessage(), code)
pdfControl	処理18	int code = e.getStatusCode()
pdfControl	処理19	continue
pdfControl	処理20	agent = null
pdfControl	処理21	session.invalidate()
pdfControl	条件22	_timeout > 0
pdfControl	処理23	agent = new SessionAgent(session.getId(), null, null)
pdfControl	処理24	agent.stopTimer()
pdfControl	処理25	agent = (SessionAgent)session.getAttribute(AGENTNAME)
pdfControl	処理26	this.returnHttpErrorResponse(response, null, "INTERNAL ERROR (session tracking error).", STATUS_ERROR)
pdfControl	処理27	session = request.getSession(true)

イベント	参照番号	原本条件・処理
pdfControl	処理28	HttpSession session = null
pdfControl	処理29	SessionAgent agent = null
pdfControl	処理30	SessionAgent agent = null
pdfControl	処理31	HttpSession session = null
pdfControl	処理32	session = request.getSession(true)
pdfControl	処理33	this.returnHttpErrorResponse(response, null, "INTERNAL ERROR (session tracking error).", STATUS_ERROR)
pdfControl	処理34	agent = (SessionAgent)session.getAttribute(AGENTNAME)
pdfControl	処理35	agent = new SessionAgent(session.getId(), null, null)
pdfControl	条件36	_timeout > 0
pdfControl	処理37	agent.stopTimer()
pdfControl	処理38	int code = e.getStatusCode()
pdfControl	処理39	this.returnHttpErrorResponse(response, null, e.getMessage(), code)
pdfControl	処理40	session.invalidate()
pdfControl	処理41	session.invalidate()
pdfControl	処理42	agent = null
pdfControl	処理43	continue
pdfControl	処理44	FMResponse result = null
pdfControl	処理45	agent.parseParameters(request)

イベント	参照番号	原本条件・処理
pdfControl	処理46	agent.clear()
pdfControl	処理47	result = agent.getLastResponse()
pdfControl	条件48	agent.canOverlay()
pdfControl	処理49	result = agent.overlay()
pdfControl	処理50	String msg = agent.getMessage()
pdfControl	処理51	int code = e.getStatusCode()
pdfControl	処理52	this.returnHttpErrorResponse(response, null, e.getMessage(), code)
pdfControl	処理53	agent.startTimer()
pdfControl	処理54	int code = e.getStatusCode()
pdfControl	処理55	this.returnHttpErrorResponse(response, null, e.getMessage(), code)
pdfControl	処理56	agent.startTimer()
pdfControl	処理57	this.returnHttpResponse(response, agent, result, method)
pdfControl	処理58	agent.startTimer()
pdfControl	条件59	FMSystemException e
pdfControl	条件60	FMPropertyException e
pdfControl	条件61	agent
pdfControl	条件62	agent
pdfControl	条件63	Exception e

イベント	参照番号	原本条件・処理
pdfControl	条件64	agent
pdfControl	条件65	agent
pdfControl	条件66	agent.canOverlay()
pdfControl	条件67	agent
pdfControl	条件68	NOT (agent.canOverlay())
pdfControl	条件69	agent
pdfControl	条件70	FMPrintException e
pdfControl	条件71	agent
pdfControl	条件72	FMSystemException e
pdfControl	条件73	agent
pdfControl	条件74	FMPropertyException e
pdfControl	条件75	agent
pdfControl	条件76	FMSystemException e
pdfControl	条件77	agent
pdfControl	条件78	Exception e
pdfControl	条件79	agent
handleTimer	処理1	e.printStackTrace()
handleTimer	処理2	session.removeAttribute(AGENTNAME)
handleTimer	処理3	agent.invalidate()

イベント	参照番号	原本条件・処理
handleTimer	処理4	SessionAgent agent = (SessionAgent)session.getAttribute(AGENTNAME)
handleTimer	処理5	HttpSession session = (HttpSession)userInfo
handleTimer	処理6	HttpSession session = (HttpSession)userInfo
handleTimer	処理7	SessionAgent agent = (SessionAgent)session.getAttribute(AGENTNAME)
handleTimer	処理8	agent.invalidate()
handleTimer	処理9	session.removeAttribute(AGENTNAME)
handleTimer	処理10	e.printStackTrace()
setContentLength	処理1	length = data.length
setContentLength	処理2	length = file.length()
setContentLength	処理3	File file = (File) overlayRes.get("fm-datafile")
setContentLength	処理4	byte[] data = (byte[]) overlayRes.get("fm-databytes")
setContentLength	処理5	long length = 0
setContentLength	処理6	long length = 0
setContentLength	処理7	byte[] data = (byte[]) overlayRes.get("fm-databytes")
setContentLength	処理8	length = data.length
setContentLength	処理9	File file = (File) overlayRes.get("fm-datafile")
setContentLength	処理10	length = file.length()
returnHttpErrorResponse	処理1	e.printStackTrace()

イベント	参照番号	原本条件・処理
returnHttpErrorResponse	処理2	errout.flush()
returnHttpErrorResponse	処理3	errout.println("</HTML>")
returnHttpErrorResponse	処理4	errout.println("</BODY>")
returnHttpErrorResponse	処理5	errout.println("<H2>Error Code : " + code +"</H2>")
returnHttpErrorResponse	処理6	errout.println("<H2>Error Message : " + message +"</H2>")
returnHttpErrorResponse	処理7	errout.println("<BODY>")
returnHttpErrorResponse	処理8	errout.println("</HEAD>")
returnHttpErrorResponse	処理9	errout.println("<HEAD><TITLE>ERROR</TITLE>")
returnHttpErrorResponse	処理10	errout.println("<HTML>")
returnHttpErrorResponse	処理11	PrintWriter errout = response.getWriter()
returnHttpErrorResponse	処理12	PrintWriter errout = response.getWriter()
returnHttpErrorResponse	処理13	errout.println("<HTML>")
returnHttpErrorResponse	処理14	errout.println("<HEAD><TITLE>ERROR</TITLE>")
returnHttpErrorResponse	処理15	errout.println("</HEAD>")
returnHttpErrorResponse	処理16	errout.println("<BODY>")
returnHttpErrorResponse	処理17	errout.println("<H2>Error Message : " + message +"</H2>")
returnHttpErrorResponse	処理18	errout.println("<H2>Error Code : " + code +"</H2>")
returnHttpErrorResponse	処理19	errout.println("</BODY>")
returnHttpErrorResponse	処理20	errout.println("</HTML>")

イベント	参照番号	原本条件・処理
returnHttpErrorResponse	処理21	errout.flush()
returnHttpErrorResponse	処理22	e.printStackTrace()